Project 1: CSR SpMM: Profiling the Impact of Memory Access Locality

1. Program Self Test

Symbol or array	Definition
R	Number of rows in sparse matrix A
C	Number of columns in sparse matrix A and rows in dense matrix X
NNZ	Number of nonzero entries in A
F	Number of features per row of X and Y
A	Sparse $R \times C$ matrix in CSR format
X	Dense $C \times F$ row-major feature matrix
Y	Dense $R \times F$ row-major output matrix
`row_ptr`	`std::size_t` row offsets with $R + 1$ entries
`col_idx`	`std::uint32_t` column indices with NNZ entries
`values`	`double` nonzero weights with NNZ entries

The self test computes the following small matrix product:

$$\mathbf{A} = \begin{bmatrix} 2 & 0 & 0 & 1 \\ 0 & -1 & 3 & 0 \\ 4 & 0 & 0 & 0 \end{bmatrix}, \qquad \mathbf{X} = \begin{bmatrix} 1 & 2 \\ 2 & 1 \\ 3 & 0 \\ 4 & -1 \end{bmatrix}.$$

$$\begin{aligned} Y_{0,:} &= 2X_{0,:} + 1X_{3,:} = [6, 3], \\ Y_{1,:} &= -1X_{1,:} + 3X_{2,:} = [7, -1], \\ Y_{2,:} &= 4X_{0,:} = [4, 8]. \end{aligned}$$

Thus, the expected result is:

$$\mathbf{Y} = \begin{bmatrix} 6 & 3 \\ 7 & -1 \\ 4 & 8 \end{bmatrix}.$$

2. Local vs Permuted Memory Layouts

```
$ ./build/spmm_gnn --pattern local --rows 32768 --nnz-per-row 8 --features
32 --iters 100
$ ./build/spmm_gnn --pattern permuted --rows 32768 --nnz-per-row 8 --
features 32 --iters 100
```

Both runs differ only in memory access locality. In the `local` layout, logical column IDs are stored in their original order, so neighboring accesses to X are more likely to be close in memory. In the `permuted` layout, the column IDs and corresponding feature rows are deterministically scrambled together, which reduces access locality. The benchmark times only repeated kernel execution after setup.

Result	Local	Permuted
Setup time (s)	0.024224	0.048431
Timed kernel (s)	0.296643	0.605645
Time per iteration (ms)	2.966428	6.056448
Time per edge-feature (ns)	0.353626	0.721985

3. Calculate Memory Footprint for a Test Case

```
$ python3 src/memory_footprint.py
```

The test case uses $R = 32768$, $C = 4R = 131072$, $F = 32$, and 8 stored entries per row, so $NNZ = R \times 8 = 262144$.

Each array's footprint is its element count multiplied by its element size:

Array	Element type	Footprint formula (bytes)	Size
`row_ptr`	`std::size_t`	$(R+1) \times 8$	0.25 MiB
`col_idx`	`std::uint32_t`	$NNZ \times 4$	1.00 MiB
`values`	`double`	$NNZ \times 8$	2.00 MiB
`X`	`double`	$C \times F \times 8$	32.00 MiB
`Y`	`double`	$R \times F \times 8$	8.00 MiB
`expected`	`double`	$R \times F \times 8$	8.00 MiB
Total	—	sum of the above	**51.25 MiB**

Here, sizes count array payloads only, excluding vector metadata and allocator overhead.

4. Traffic vs Storage

Storage is the space occupied by the arrays, independent of how often their elements are accessed. X occupies 32.00 MiB, and the six array payloads total 51.25 MiB, as calculated above. The actual allocation footprint can also include **vector metadata, allocator overhead, and alignment padding**, which the payload calculation excludes.

Traffic counts data movement during execution. Each stored entry of A causes the kernel to logically read F consecutive `double` values from X, in both local and permuted layouts. Therefore, one complete `csr_spmm` invocation has logical feature read demand:

For example, the logical read demand is calculated as:

$$B_{\text{logical feature reads}} = NNZ \times F \times 8 = 262144 \times 32 \times 8 = 67{,}108{,}864 \text{ bytes} = 64 \text{ MiB}.$$

Logical read demand is not measured DRAM traffic. The distinction depends on several effects:

- **Cache hits and reuse:** repeated reads may be served by cache, producing no additional DRAM reads. Data can also remain cached between timed invocations.
- **Cache misses and eviction:** capacity limits and cache conflicts can cause the same data to be fetched from DRAM multiple times.
- **Cache-line granularity and alignment:** memory is fetched in cache-line units, so a transfer can include bytes beyond those requested. Contiguous features help use the fetched bytes.
- **Prefetching:** hardware may fetch data before it is requested, including data that is never used.
- **Other memory accesses and write overhead:** the kernel also reads CSR arrays and writes Y. Depending on the cache policy, output stores can cause write allocation and later dirty-line writebacks. These are absent from the X-only demand formula. The allocated `expected` array is used for validation, rather than read by `csr_spmm`.
- **Synchronization and coherence:** in a parallel implementation, locks, atomics, and cache-line sharing can add communication and execution overhead without changing array sizes. This kernel is single-threaded and has no explicit thread synchronization; cache-to-cache transfers would also need to be distinguished from DRAM transfers.

Both local and permuted layouts have the same storage footprint and logical feature read demand. Their DRAM traffic may differ because their memory access locality differs.

5. Access Pattern Interpretation

For one nonzero entry $A_{r,c}$, the kernel reads all F features in the contiguous row $X[c, :]$. With row-major storage, these feature values are adjacent, so one cache-line fetch can serve several accesses. Nearby edges that reference nearby rows of X therefore benefit from better spatial locality in the local layout.

The benefit is not guaranteed. For example, if the feature table and the set of accessed rows are much larger than the cache, or if the permuted access pattern provides little temporal reuse, each row of X may be evicted before it is reused. In that case, most accesses can miss in cache even though the F features within each row are contiguous.

6. Propose a Falsifiable Reuse Hypothesis

hypothesis: for the fixed baseline workload, the local layout will have lower kernel time and lower time per edge-feature than the permuted layout because nearby neighbor references reuse cache lines from nearby rows of x. the implementation, input, machine state, timed region, correctness checks, and statistical procedure must remain fixed; both layouts must perform identical work and pass the same oracle; and performance evidence must distinguish memory pressure from front-end or core-bound limitations.

Falsification experiment: Compile once, then run `local` and `permuted` with identical dimensions, iteration counts, warm-up, timed kernel boundaries, and correctness checks. Repeat each run and compare `ms_per_iter` and `ns_per_edge_feature`. The hypothesis is falsified if repeated measurements show no consistent performance advantage for the local layout. PMU counters, if collected, provide supporting evidence rather than a direct DRAM-traffic measurement.